



Module 15 — Pandas approfondi + DuckDB

Bootcamp Data Analyst — From Zero to Hero | Niveau Intermédiaire · Module 15



Ce que tu seras capable de faire à la fin de ce module

- Distinguer `transform` / `mutate` (garde une ligne par observation) de `agg` / `summarise` (résume par groupe)
- Découper une variable continue en tranches (`pd.cut` / `cut()`)
- Écrire une analyse lisible en chaînant les étapes (method chaining / pipe `%>%`)
- Interroger un DataFrame directement en SQL avec **DuckDB**, en Python et en R
- Savoir quand utiliser DuckDB plutôt qu'une vraie base de données (Module 16)



Durée estimée : 2 heures  **Outils** : Python (`pandas`, `duckdb`) · R (`dplyr`, `duckdb`)



Prérequis : Modules 10-12 (Niveau Débutant) · Module 14

1. Pourquoi aller au-delà du `groupby` / `group_by` simple

Au Niveau Débutant, tu as appris à grouper et résumer : moyenne par département, comptage par catégorie. En situation réelle, trois besoins reviennent sans cesse et méritent des outils dédiés :

1. **Comparer une valeur à la moyenne de son groupe**, sans perdre le détail ligne par ligne (ex : "de combien ce salaire s'écarte-t-il de la moyenne de son département ?")

2. Découper une variable continue en catégories lisibles (ex : ancienneté en "Junior / Confirmé / Senior" plutôt qu'un nombre d'années brut)
3. Interroger rapidement un DataFrame avec du SQL, sans monter une base de données, quand tu préfères la syntaxe `SELECT` à `groupby`

Le dataset — on retrouve celui du Module 11

Même jeu de données RH de **Bamba & Associés** qu'au Module 11 (15 employés) — mais cette fois avec des techniques plus avancées.

NOM	DÉPARTEMENT	CONTRAT	SALAIRE (FCFA)	ANCIENNETÉ (ANS)
Kouassi Ama	RH	CDI	450 000	5
Kouamé Jean	Finance	CDI	620 000	8
Traoré Seydou	Commercial	CDI	380 000	3
Diabaté Fatou	RH	CDD	310 000	1
Bamba Koné	Direction	CDI	1 800 000	12
Ouédraogo Moussa	IT	CDI	580 000	6
Soro Aminata	Commercial	Stage	180 000	0
Koné Ibrahim	Finance	CDI	540 000	7
Yao Adjoua	IT	CDI	610 000	9
Diallo Mamadou	Commercial	CDD	340 000	2
Coulibaly Assata	RH	CDI	420 000	4
N'Guessan Éric	Finance	CDI	590 000	7
Touré Mariam	IT	Stage	200 000	0
Keita Souleymane	Commercial	CDI	360 000	3
Fofana Raïssa	Direction	CDI	950 000	10

2. `transform` / `mutate` vs `agg` / `summarise`

C'est la distinction la plus utile de ce module : est-ce que tu veux **garder une ligne par employé** (avec une info de groupe ajoutée), ou **résumer en une ligne par groupe** ?

Python (`transform` garde les 15 lignes) :

PYTHON

```
df["salaire_moyen_dept"] = df.groupby("departement")
["salaire"].transform("mean").round(0)
print(df[["nom", "departement", "salaire", "salaire_moyen_dept"]].head(4))
```

#	nom	departement	salaire	salaire_moyen_dept
# 0	Kouassi Ama	RH	450000	393333.0
# 1	Kouamé Jean	Finance	620000	583333.0
# 2	Traoré Seydou	Commercial	380000	315000.0
# 3	Diabaté Fatou	RH	310000	393333.0

Python (`agg`, collapse à 5 lignes — une par département) :

PYTHON

```
print(df.groupby("departement")["salaire"].mean().round(0))
```

#	departement	
#	Commercial	315000.0
#	Direction	1375000.0
#	Finance	583333.0
#	IT	463333.0
#	RH	393333.0

R (`mutate`, équivalent de `transform` — garde les 15 lignes) :

R

```
library(dplyr)

df <- df %>%
  group_by(departement) %>%
  mutate(salaire_moyen_dept = round(mean(salaire), 0)) %>%
  ungroup()

df %>% select(nom, departement, salaire, salaire_moyen_dept) %>% head(4)
```

R (`summarise`, équivalent de `agg` — collapse par groupe) :

```
R
df %>%
  group_by(departement) %>%
  summarise(salaire_moyen = round(mean(salaire), 0))
```

🔗 **Pont :** `transform` (Python) = `mutate` (R) → une ligne par observation, une colonne de groupe en plus. `agg` (Python) = `summarise` (R) → une ligne par groupe, le détail individuel disparaît.

À quoi ça sert concrètement

```
PYTHON
df["ecart_moyenne"] = (df["salaire"] - df["salaire_moyen_dept"]).round(0)
print(df[["nom", "departement",
"ecart_moyenne"]].sort_values("ecart_moyenne", ascending=False).head(3))
#               nom departement  ecart_moyenne
# 4      Bamba Koné  Direction      425000.0
# 8      Yao Adjoua           IT       146667.0
# 5  Ouédraogo Moussa           IT       116667.0
```

Bamba Koné gagne 425 000 FCFA de plus que la moyenne de son département — une info impossible à obtenir avec un simple `agg`, puisqu'elle nécessite de garder chaque ligne individuelle **et** la moyenne de son groupe côte à côte.

3. Découper une variable continue en tranches

Un nombre d'années brut ("5", "8", "0"...) est moins lisible dans un rapport que des catégories métier.

Python (`pd.cut`) :

PYTHON

```
import numpy as np

df["tranche_anciennete"] = pd.cut(
    df["anciennete"],
    bins=[-1, 2, 5, np.inf],
    labels=["Junior (0-2 ans)", "Confirmé (3-5 ans)", "Senior (6+ ans)"]
)
print(df["tranche_anciennete"].value_counts())
# tranche_anciennete
# Senior (6+ ans)      7
# Junior (0-2 ans)     4
# Confirmé (3-5 ans)   4
```

R (`cut()`, même logique de bornes) :

R

```
df$tranche_anciennete <- cut(
  df$anciennete,
  breaks = c(-1, 2, 5, Inf),
  labels = c("Junior (0-2 ans)", "Confirmé (3-5 ans)", "Senior (6+ ans)")
)
table(df$tranche_anciennete)
```

💡 Les bornes sont **exclues à gauche, incluses à droite** par défaut dans les deux langages : `(-1, 2]` inclut 0, 1 et 2, mais pas -1. Vérifie toujours un cas limite (ici, quelqu'un avec exactement 2 ans d'ancienneté) pour confirmer de quel côté il tombe.

Alternative R plus flexible — `case_when` (utile si la logique de découpage n'est pas une simple suite de bornes) :

R

```
df <- df %>%
  mutate(tranche_anciennete = case_when(
    anciennete <= 2 ~ "Junior (0-2 ans)",
    anciennete <= 5 ~ "Confirmé (3-5 ans)",
    TRUE ~ "Senior (6+ ans)"
  ))
```

4. Écrire une analyse lisible — method chaining / pipe

Plutôt que d'empiler des variables intermédiaires (`etape1 = ...`, `etape2 = ...`), on **chaîne** les opérations pour lire l'analyse comme une phrase.

Python :

PYTHON

```
resultat = (
    df.groupby("departement")["salaire"]
    .agg(["mean", "count"])
    .round(0)
    .rename(columns={"mean": "salaire_moyen", "count": "effectif"})
    .sort_values("salaire_moyen", ascending=False)
)
print(resultat)
```

	salaire_moyen	effectif
# Direction	1375000.0	2
# Finance	583333.0	3
# IT	463333.0	3
# RH	393333.0	3
# Commercial	315000.0	4

R (avec le pipe `%>%`, identique dans l'esprit) :

R

```
resultat <- df %>%
  group_by(departement) %>%
  summarise(salaire_moyen = round(mean(salaire), 0), effectif = n()) %>%
  arrange(desc(salaire_moyen))

print(resultat)
```

 **Pont** : les parenthèses `(...)` en Python et le pipe `%>%` en R jouent le même rôle — enchaîner les étapes sans variables intermédiaires jetables. Le R moderne propose aussi le pipe natif `|>` (équivalent à `%>%` pour un usage simple).

5. DuckDB — interroger un DataFrame en SQL

DuckDB est un moteur SQL "embarqué" (in-process) — pas de serveur à lancer, pas de connexion réseau. Il s'installe comme une librairie et peut interroger directement un DataFrame Python/R avec la syntaxe SQL que tu maîtrises depuis les Modules 06-09.

Python

PYTHON

```
import duckdb

resultat = duckdb.sql("""
    SELECT departement, ROUND(AVG(salaire),0) AS salaire_moyen, COUNT(*) AS
effectif
    FROM df
    WHERE contrat = 'CDI'
    GROUP BY departement
    ORDER BY salaire_moyen DESC
""").df()
print(resultat)
#  departement  salaire_moyen  effectif
# 0  Direction      1375000.0         2
# 1         IT       595000.0         2
# 2     Finance      583333.0         3
# 3         RH       435000.0         2
# 4  Commercial      370000.0         2
```

DuckDB reconnaît automatiquement `df` comme une table — pas besoin de l'importer ou de le déclarer, tant que la variable existe dans ton environnement Python.

R

```
R
library(duckdb)
library(DBI)

con <- dbConnect(duckdb())
duckdb_register(con, "df", df)

resultat <- dbGetQuery(con, "
  SELECT departement, ROUND(AVG(salaire),0) AS salaire_moyen, COUNT(*) AS
effectif
  FROM df
  WHERE contrat = 'CDI'
  GROUP BY departement
  ORDER BY salaire_moyen DESC
")
print(resultat)

dbDisconnect(con, shutdown = TRUE)
```

En R, il faut explicitement **enregistrer** le DataFrame (`duckdb_register`) avant de pouvoir le requêter, et **fermer proprement la connexion** (`dbDisconnect(..., shutdown = TRUE)`) à la fin — Python gère ça de façon plus transparente.

💡 **Pourquoi ne pas juste utiliser `groupby` / `group_by` ?** Parce que si tu es plus à l'aise en SQL, ou que ta logique d'analyse est déjà écrite en SQL (par exemple copiée depuis le Module 16), DuckDB te permet de la réutiliser telle quelle sur un DataFrame local — sans base de données, sans jointure de schéma à gérer.

6. Pont DuckDB → PostgreSQL

Tu vas rencontrer un vrai PostgreSQL hébergé au Module 16. Ne confonds pas les deux :

DUCKDB (CE MODULE)		POSTGRESQL (MODULE 16)
Où ça tourne	En local, dans ton processus Python/R	Sur un serveur distant
Pour quoi faire	Prototypage rapide, analyse solo sur des fichiers/DataFrames	Stockage centralisé, partagé, auquel Power BI se connecte (Module 17)
Persistance	Éphémère (le temps du script), ou un simple fichier <code>.duckdb</code>	Persistant, multi-utilisateurs, accessible à toute l'équipe
Installation	<code>pip install duckdb</code> / <code>install.packages("duckdb")</code> — zéro serveur	Compte hébergé (Neon/Supabase) — zéro install non plus, mais un vrai serveur existe derrière
Concurrence	Un seul processus à la fois	Plusieurs utilisateurs/applications en simultané

🔗 **Retiens l'essentiel** : DuckDB pour l'analyse locale ultra-rapide et le prototypage, PostgreSQL pour le stockage persistant et centralisé de l'entreprise. Ce n'est pas "lequel est meilleur", mais "lequel correspond à ce que tu es en train de faire" — un·e DA utilise souvent les deux dans la même semaine.

✅ Résumé du module

CONCEPT	PYTHON	R
Ajouter une stat de groupe en gardant chaque ligne	<code>.groupby(...).transform(...)</code>	<code>group_by(...) %>% mutate(...)</code>
Résumer une ligne par groupe	<code>.groupby(...).agg(...)</code>	<code>group_by(...) %>% summarise(...)</code>
Découper en tranches	<code>pd.cut(col, bins=..., labels=...)</code>	<code>cut(col, breaks=..., labels=...)</code>
Découpage conditionnel flexible	<code>np.select</code> / conditions imbriquées	<code>case_when(...)</code>
Chaîner des étapes	Parenthèses <code>(...)</code> + méthodes <code>.</code>	Pipe <code>%>%</code> ou <code>`</code>
SQL sur un DataFrame	<code>duckdb.sql("... FROM df")</code>	<code>dbGetQuery(con, "... FROM df")</code> après <code>duckdb_register</code>
DuckDB vs PostgreSQL	Local, prototypage	Distant, centralisé, multi-utilisateurs

Quiz — Vérifie ta compréhension

Q1. Tu veux ajouter une colonne "salaire moyen du département" à ton DataFrame, en gardant les 15 lignes individuelles. Tu utilises...

- a) `.groupby("departement")["salaire"].agg("mean")`
- b) `.groupby("departement")["salaire"].transform("mean")`
- c) `.groupby("departement")["salaire"].sum()`

Voir la réponse

✓ **b)** — `transform` renvoie un résultat de la **même taille** que la colonne d'origine (une valeur par ligne), donc tu peux l'assigner directement comme nouvelle colonne. `agg` (a) collapse en une ligne par groupe — impossible à réassigner ligne par ligne sans jointure supplémentaire.

Q2. En R, quel est l'équivalent de `.groupby(...).transform(...)` ?

- a) `group_by(...) %>% summarise(...)`
- b) `group_by(...) %>% mutate(...)`
- c) `group_by(...) %>% filter(...)`

👉 Voir la réponse

✅ **b)** — `mutate` ajoute une colonne en gardant une ligne par observation, exactement comme `transform` en Python. `summarise` (a) est l'équivalent de `agg` (collapse par groupe).

Q3. Pourquoi choisir DuckDB plutôt qu'un vrai PostgreSQL pour analyser un fichier CSV que tu viens de télécharger ?

- a) DuckDB est plus rapide dans tous les cas, il faut toujours le préférer
- b) Le fichier est local et l'analyse est ponctuelle — pas besoin d'un serveur centralisé et partagé
- c) PostgreSQL ne sait pas faire de `GROUP BY`

👉 Voir la réponse

✅ **b)** — DuckDB convient au prototypage local sur un fichier que toi seul·e utilises. Dès que plusieurs personnes ou outils (comme Power BI) doivent accéder aux mêmes données en continu, il faut un serveur centralisé comme PostgreSQL.

➡ Module suivant

Tu sais maintenant manipuler des DataFrames en profondeur et les interroger en SQL local. Direction le **Module 16**, où tu montes un vrai PostgreSQL hébergé et écris des requêtes avancées.

